CTM: Conv-Temporal-Mamba Architecture

A Comprehensive Guide for Stock Prediction Systems

Quantitative Research

May 22, 2026

Contents

1 Executive Summary

The Conv-Temporal-Mamba (CTM) architecture combines the **selective state space model** (Mamba) with **causal convolutional processing** and **temporal feature engineering** for stock return prediction. Unlike Transformer-based approaches that scale quadratically with sequence length, CTM achieves linear $\mathcal{O}(T)$ complexity while matching or exceeding prediction accuracy.

This guide provides:

- A complete mathematical specification of the CTM architecture
- PyTorch implementation patterns derived from state-of-the-art research
- Quantitative loss functions optimized for trading metrics
- Scaling strategies from single-stock to multi-asset portfolios
- Verification and production deployment guidelines

Key references:

- Mamba: *Linear-Time Sequence Modeling with Selective State Spaces* (Gu & Dao, 2023)
- MambaStock: *Selective State Space Model for Stock Prediction* (Shi, 2024)
- S-Mamba: *Is Mamba Effective for Time Series Forecasting?* (Wang et al., 2025)
- SAMBA: *Mamba Meets Financial Markets via Graph Convolution* (Mehrabian et al., 2025)
- DMamba: *Decomposition-enhanced Mamba for Time Series* (Chen & Sun, 2026)
- Mamba-ProbTSF: *Uncertainty Quantification in SSM Forecasts* (Pessoa et al., 2025)

2 Motivation: Why Mamba for Stock Prediction?

Time series forecasting is the backbone of quantitative finance. Table 1 compares the three dominant sequence modeling paradigms.

Table 1: Architecture comparison for financial time series

Property	LSTM	Transformer	**Mamba (SSM)**
Train complexity	$\mathcal{O}(T)$	$\mathcal{O}(T^2)$	$\mathcal{O}(T)$
Inference per step	$\mathcal{O}(1)$	$\mathcal{O}(T)$	$\mathcal{O}(1)$
Long-range memory	Poor	Excellent	Excellent
Parallel training	Sequential	Full	Full (associative scan)
Gradient stability	Moderate	Good	**Excellent** (diag. A)
Financial SOTA	Baseline	Strong	**New SOTA**

2.1 The Selective SSM Advantage

The core innovation of Mamba is the **selective scan** mechanism (S6). Standard SSMs (`S4`, `S4D`) have input-*independent* transition matrices. Mamba makes Δ_t, $\mathbf{B}_t$, and $\mathbf{C}_t$ functions of the input $\mathbf{x}_t$, enabling *content-based selection*.

Bi-Mamba Safety Note: All Bi-Mamba processing operates exclusively on the historical lookback window $\{\mathbf{x}_{t-T+1}, \ldots, \mathbf{x}_t\}$ where $\mathbf{x}_t$ is the most recent *observed* data point. No future data beyond the input window is accessed. See Section 5.3 for streaming inference implications.

The financial intuition is direct:

Δ_t **(step size)** Controls adaptation speed. Large Δ during news events (rapid regime change); small Δ during calm periods (smooth filtering).

$\mathbf{B}_t$ **(input gate)** Selects which market signals enter the latent state. Suppresses noise during low-volatility regimes; amplifies predictive signals at turning points.

$\mathbf{C}_t$ **(output gate)** Controls which parts of the learned market state are read for prediction. The model can learn trend-following components in bull markets and mean-reversion in range-bound markets.

$\overline{\mathbf{A}}_t = \exp(\Delta_t \odot \mathbf{A})$ **(state decay)** The "memory half-life." Near 1 means long memory (trend regime); near 0 means short memory (mean-reversion).

2.2 Empirical Evidence

Recent studies consistently show Mamba-based models outperforming LSTM and Transformer baselines on financial data:

Table 2: Financial Mamba empirical results

Paper	Data	Best Model	vs. Best Baseline
MambaStock (2024)	4 Chinese A-shares	Mamba (2-layer)	MAE -18%
CryptoMamba (2025)	Bitcoin multi-regime	Bi-Mamba	RMSE -22%
SAMBA (2025)	US equities (82 feat.)	Graph+Mamba	exceeds SOTA
CrossMamba (2026)	S&P 500	Mamba-Aug. Trans.	$R^2 = 0.963$

3 CTM Architecture Overview

The CTM architecture extends vanilla Mamba with three enhancements:

1. **Conv:** Causal 1D convolution captures local temporal patterns before the SSM, serving as a learned feature preprocessor.
2. **Temporal:** Multi-scale temporal decomposition separates trend and seasonal components for specialized processing.
3. **Mamba:** Selective SSM backbone provides linear-complexity long-range sequence modeling.

3.1 Architecture Variants

Based on our research, we identify five validated architecture patterns:

Pattern 1: Vanilla Mamba Stack (MambaStock baseline) 2–4 Mamba layers with residual connections. Uses only selective scan with SiLU gating. This is the simplest proven configuration for stock prediction.

Pattern 2: Decomposition + Specialized Modules (DMamba)

$$\text{Input} \to \text{Seasonal-Trend Decomp} \to \begin{cases} \text{Seasonal: Mamba (high-dim)}, \\ \text{Trend: MLP (low-dim)} \end{cases} \to \text{Fuse} \to \text{Predict}$$

Pattern 3: Bidirectional Mamba (S-Mamba, SAMBA) Processes both forward and backward passes through separate Mamba stacks, then concatenates features before the prediction head.

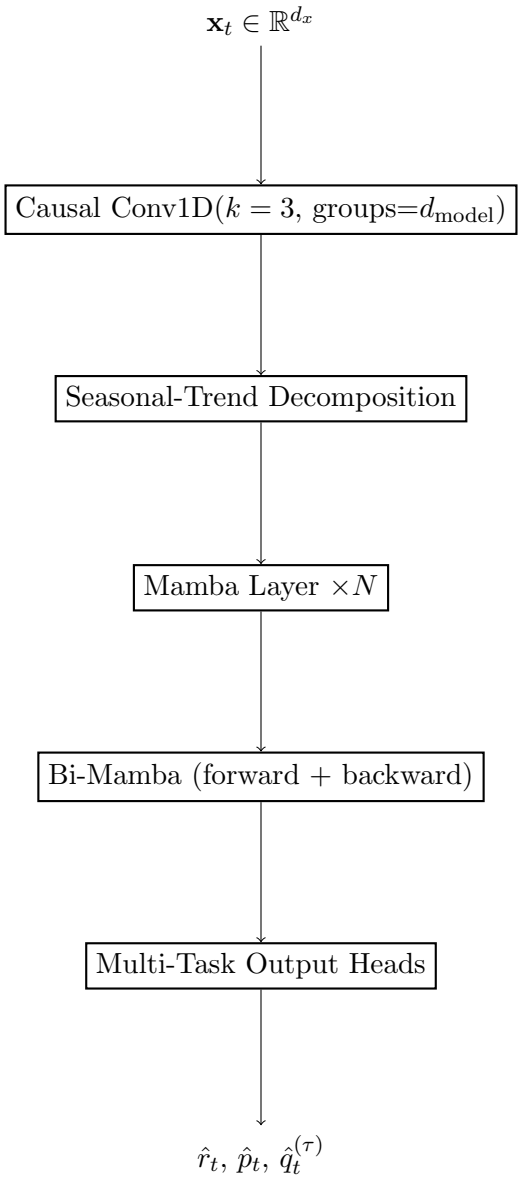

Figure 1: CTM architecture overview

Pattern 4: Graph + Mamba (SAMBA) For multi-stock prediction, adds a Chebyshev graph convolution layer after Mamba to model inter-stock correlations with a learned adjacency matrix.

Pattern 5: Hybrid Mamba-Transformer (CrossMamba) Interleaves Mamba and attention layers, where Mamba handles long-range dependencies and attention focuses on local patterns.

4 Mamba Core: Selective State Space Model

4.1 Mathematical Formulation

A continuous state space model maps an input signal $\mathbf{x}(t) \in \mathbb{R}$ to an output $\mathbf{y}(t) \in \mathbb{R}$ through a latent state $\mathbf{h}(t) \in \mathbb{R}^N$:

$$\mathbf{h}'(t) = \mathbf{A}\mathbf{h}(t) + \mathbf{B}\mathbf{x}(t) \tag{1}$$
$$\mathbf{y}(t) = \mathbf{C}\mathbf{h}(t) + \mathbf{D}\mathbf{x}(t) \tag{2}$$

where $\mathbf{A} \in \mathbb{R}^{N\times N}$ is the state transition matrix, $\mathbf{B} \in \mathbb{R}^{N\times 1}$ the input matrix, $\mathbf{C} \in \mathbb{R}^{1\times N}$ the output matrix, and $\mathbf{D} \in \mathbb{R}$ the skip connection.

For discrete sequences $\{\mathbf{x}_1, \ldots, \mathbf{x}_T\}$ with step size Δ, the zero-order hold (ZOH) discretization gives:

$$\overline{\mathbf{A}} = \exp(\Delta\mathbf{A}) \tag{3}$$
$$\overline{\mathbf{B}} = \mathbf{A}^{-1}(\exp(\Delta\mathbf{A}) - \mathbf{I}) \cdot \mathbf{B} \approx \Delta\mathbf{B} \tag{4}$$

where $\mathbf{A}^{-1}(\exp(\Delta\mathbf{A}) - \mathbf{I})\mathbf{B}$ is the exact ZOH and $\Delta\mathbf{B}$ is the Euler approximation (which Mamba uses in practice for efficiency). The discrete recurrence is:

$$\mathbf{h}_t = \overline{\mathbf{A}}\mathbf{h}_{t-1} + \overline{\mathbf{B}}\mathbf{x}_t \tag{5}$$
$$\mathbf{y}_t = \mathbf{C}\mathbf{h}_t + \mathbf{D}\mathbf{x}_t \tag{6}$$

4.2 Selective Scan (S6)

Mamba makes Δ_t, $\mathbf{B}_t$, and $\mathbf{C}_t$ functions of the input $\mathbf{x}_t$. Given projection weights $\mathbf{W}_\Delta$, $\mathbf{W}_B$, $\mathbf{W}_C$:

$$\Delta_t = \mathrm{clamp}\big(\mathrm{softplus}(\mathbf{W}_\Delta \mathbf{x}'_{\mathrm{conv},t} + \mathbf{b}_\Delta),\ 10^{-5},\ 20.0\big) \tag{7}$$
$$\mathbf{B}_t = \mathbf{W}_B \mathbf{x}'_{\mathrm{conv},t} \tag{8}$$
$$\mathbf{C}_t = \mathbf{W}_C \mathbf{x}'_{\mathrm{conv},t} \tag{9}$$

The discretization with diagonal $\mathbf{A}$ (for efficiency, $\mathbf{A} = \mathrm{diag}(a_1, \ldots, a_{d_{\mathrm{model}}})$) becomes element-wise:

$$\overline{\mathbf{A}}_t = \exp\left(\Delta_t \odot \mathbf{A}\right) \tag{10}$$
$$\overline{\mathbf{B}}_t = \Delta_t \otimes \mathbf{B}_t^\top \quad \text{(outer product: } \overline{B}_t[c,s] = \Delta_t[c] \cdot B_t[s]\text{)} \tag{11}$$

The selective scan recurrence per channel i:

$$h_t[i] = \overline{A}_t[i] \cdot h_{t-1}[i] + \overline{\mathbf{B}}_t[i,:] \cdot x'_{\mathrm{conv},t}[i] \tag{12}$$
$$y_{\mathrm{ssm},t}[i] = \mathbf{C}_t^\top h_t[i] + D[i] \cdot x'_{\mathrm{conv},t}[i] \tag{13}$$

4.3 Matrix Implementation

In practice, all channels are processed simultaneously using batched operations:

$$\mathbf{h}_t = \overline{\mathbf{A}}_t \odot \mathbf{h}_{t-1} + \overline{\mathbf{B}}_t \odot \mathbf{x}'_{\mathrm{conv},t} \quad \text{(broadcast)} \tag{14}$$
$$\mathbf{y}_{\mathrm{ssm},t} = \mathbf{C}_t^\top \mathbf{h}_t + \mathbf{D} \odot \mathbf{x}'_{\mathrm{conv},t} \tag{15}$$

where $\mathbf{h}_t \in \mathbb{R}^{d_{\mathrm{model}}\times d_{\mathrm{state}}}$, $\overline{\mathbf{A}}_t \in \mathbb{R}^{d_{\mathrm{model}}\times 1}$, $\overline{\mathbf{B}}_t \in \mathbb{R}^{d_{\mathrm{model}}\times d_{\mathrm{state}}}$.

Algorithm 1 Mamba block forward pass (per time step t)

$\mathbf{z} \leftarrow \mathbf{W}_{\text{in}}\mathbf{x}_t$ {Input projection: $(2d_{\text{model}}) \times 1$}
$\mathbf{z}_1, \mathbf{z}_2 \leftarrow \text{split}(\mathbf{z})$ {SSM branch, gate branch}
$\mathbf{x}'_{\text{conv},t} \leftarrow \text{causal\_conv1d}(\mathbf{z}_1)$ {Local feature extraction}
$\mathbf{x}'_{\text{conv},t} \leftarrow \text{SiLU}(\mathbf{x}'_{\text{conv},t})$ {Activation}
$\Delta_t \leftarrow \text{clamp}\big(\text{softplus}(\mathbf{W}_\Delta \mathbf{x}'_{\text{conv},t} + \mathbf{b}_\Delta),\ 10^{-5},\ 20.0\big)$ {Step size}
$\mathbf{B}_t \leftarrow \mathbf{W}_B \mathbf{x}'_{\text{conv},t}$ {Input matrix}
$\mathbf{C}_t \leftarrow \mathbf{W}_C \mathbf{x}'_{\text{conv},t}$ {Output matrix}
$\overline{\mathbf{A}}_t \leftarrow \exp(\Delta_t \odot \mathbf{A})$ {Discretize $\mathbf{A}$}
$\overline{\mathbf{B}}_t \leftarrow \Delta_t \odot \mathbf{B}_t^\top$ {Discretize $\mathbf{B}$}
$\mathbf{h}_t \leftarrow \overline{\mathbf{A}}_t \odot \mathbf{h}_{t-1} + \overline{\mathbf{B}}_t \odot \mathbf{x}'_{\text{conv},t}$ {SSM recurrence}
$\mathbf{y}_{\text{ssm},t} \leftarrow \mathbf{C}_t^\top \mathbf{h}_t + \mathbf{D} \odot \mathbf{x}'_{\text{conv},t}$ {SSM output}
$\mathbf{y}_t \leftarrow \text{SiLU}(\mathbf{z}_2) \odot \mathbf{y}_{\text{ssm},t}$ {Gating}
$\hat{\mathbf{r}}_t \leftarrow \mathbf{W}_{\text{out}}\mathbf{y}_t$ {Output projection}

return $\hat{\mathbf{r}}_t, \mathbf{h}_t$

4.4 Full Mamba Block Forward Pass

4.5 Parallel Scan for Training

The recurrence $\mathbf{h}_t = \overline{\mathbf{A}}_t \odot \mathbf{h}_{t-1} + \overline{\mathbf{B}}_t \odot \mathbf{x}'_{\text{conv},t}$ is an **associative operation** and can be computed in parallel using a scan algorithm:

Algorithm 2 Parallel associative scan (Blelloch, 1990)

Require: Elements $\{e_t = (\overline{\mathbf{A}}_t, \overline{\mathbf{B}}_t \odot \mathbf{x}'_{\text{conv},t})\}_{t=1}^T$
Ensure: Prefix products $\{\mathbf{h}_t\}_{t=1}^T$
{Phase 1: Up-sweep (tree reduction)}
for $d = 1$ to $\log_2 T$ **do**
  for $k = 0$ to $T - 1$ step 2^d **do**
    $e_{k+2^d-1} \leftarrow e_{k+2^{d-1}-1} \circ e_{k+2^d-1}$
  end for
end for
{Phase 2: Down-sweep}
$e_{T-1} \leftarrow$ identity {Replace last with identity}
for $d = \log_2 T - 1$ down to 0 **do**
  for $k = 0$ to $T - 1$ step 2^{d+1} **do**
    temp $\leftarrow e_{k+2^d-1}$
    $e_{k+2^d-1} \leftarrow e_{k+2^{d+1}-1} \circ$ temp
    $e_{k+2^{d+1}-1} \leftarrow$ temp $\circ\, e_{k+2^{d+1}-1}$
  end for
end for

return $\{\mathbf{h}_k\}$ extracted from e_k

The associative operator $\circ$ is defined as:

$$(a, b) \circ (a', b') = (a' \cdot a,\ a' \cdot b + b')$$

which corresponds to composing two steps of the recurrence. This reduces the sequential $\mathcal{O}(T)$ computation to $\mathcal{O}(\log T)$ parallel steps.

4.6 Mamba-2: State Space Duality (SSD)

Mamba-2 (Dao & Gu, 2024) reformulates the SSM using SSD, proving that SSMs and attention are mathematically dual. Key improvements:

- **Larger state dimension:** $d_{\text{state}} = 64$–128 (vs. 16 in Mamba-1)
- **Scalar-times-identity A matrix:** Simplified from full diagonal
- **Chunked matmul:** Divides sequence into chunks; 80–90% tensor core utilization
- **Single all-reduce:** Halves communication cost vs. Mamba-1's two all-reduces

For stock prediction, Mamba-1 with $d_{\text{state}} = 16$ is sufficient for short-mid windows ($T \leq 252$). Mamba-2 becomes relevant for very long sequences ($T > 1024$).

4.7 Mamba-3: Production Inference (March 2026)

The latest Mamba iteration introduces:

- **Exponential-Trapezoidal discretization:** Higher accuracy than ZOH
- **Complex-valued states:** Better regime change detection via RoPE
- **MIMO SSM:** Decodes multiple predictions per step without extra latency

Mamba-3 is recommended for production deployment but not necessary for prototyping.

5 Conv-Temporal Enhancement

5.1 Causal 1D Convolution

Before the SSM, a causal 1D convolution processes the input features:

$$\mathbf{x}'_{\text{conv},t} = \sum_{k=0}^{d_{\text{conv}}-1} \mathbf{W}_{\text{conv}}[:, k] \cdot \mathbf{z}_1[t-k] + \mathbf{b}_{\text{conv}}$$

where $\mathbf{z}_1[t-k] = \mathbf{0}$ when $t - k < 0$ (causal padding).

Properties:

- **Depthwise:** Groups $= d_{\text{model}}$ for parameter efficiency
- **Kernel size:** $d_{\text{conv}} = 3$–5 (small receptive field for local patterns)
- **Causal:** No look-ahead; essential for time series integrity

5.2 Multi-Scale Decomposition (DMamba Pattern)

Following DMamba (Chen & Sun, 2026), we decompose the time series before specialized processing:

$$\mathbf{x}_t = \mathbf{s}_t + \mathbf{t}_t$$

where $\mathbf{s}_t$ is the seasonal/residual component (high-frequency, high-dimensional interactions) and $\mathbf{t}_t$ is the trend component (low-frequency, modeled by a lightweight MLP).

The decomposition can be implemented via moving average:

$$\mathbf{t}_t = \frac{1}{p}\sum_{i=0}^{p-1}\mathbf{x}_{t-i} \quad \text{(trend)} \tag{16}$$

$$\mathbf{s}_t = \mathbf{x}_t - \mathbf{t}_t \quad \text{(seasonal)} \tag{17}$$

where p is the period length (e.g., $p = 5$ for weekly, $p = 21$ for monthly).

5.3 Bidirectional Mamba (S-Mamba Pattern)

For maximum temporal feature extraction, process the sequence bidirectionally. *Critical:* Both passes operate on the same historical lookback window $\{\mathbf{x}_{t-T+1}, \ldots, \mathbf{x}_t\}$ where $\mathbf{x}_t$ is the most recent observed data point. The backward pass reverses the lookback window, not the future. No look-ahead bias is introduced.

$$\mathbf{h}_t^{\rightarrow} = \text{Mamba}^{\rightarrow}(\mathbf{x}_{\leq t}) \tag{18}$$

$$\mathbf{h}_t^{\leftarrow} = \text{Mamba}^{\leftarrow}(\mathbf{x}_{t:T}) \quad \text{(reversed lookback)} \tag{19}$$

$$\mathbf{y}_t = \mathbf{W}_{\text{fuse}}\,[\mathbf{h}_t^{\rightarrow} \parallel \mathbf{h}_t^{\leftarrow}] \tag{20}$$

Streaming inference limitation: At inference time, the backward Mamba requires the full lookback window $\mathbf{x}_{1:T}$, which prevents $\mathcal{O}(1)$ per-step inference. Two options exist:

- **Batch mode:** Re-run Bi-Mamba on each new tick's lookback window (cost: $\mathcal{O}(T)$ per tick).
- **Forward-only streaming:** Use only the forward Mamba branch at inference (cached state $\mathbf{h}_{t-1}$, $\mathcal{O}(1)$ per tick). Accept the train/inference gap or train a separate forward-only model.

This mirrors the bidirectional LSTM/Transformer approach but maintains $\mathcal{O}(T)$ complexity.

6 Quantitative Loss Functions

The transition from NLP to quantitative finance requires loss functions that directly optimize trading metrics.

6.1 Mean Squared Error (MSE)

For return regression:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{T}\sum_{t=1}^{T}(r_t - \hat{r}_t)^2$$

Gradient: $\frac{\partial \mathcal{L}_{\text{MSE}}}{\partial \hat{r}_t} = -\frac{2}{T}(r_t - \hat{r}_t)$

6.2 Directional Cross-Entropy

For ternary direction prediction (UP / NEUTRAL / DOWN):

$$\mathcal{L}_{\text{Dir}} = -\frac{1}{T}\sum_{t=1}^{T}\log \hat{p}_t[y_t]$$

where $\hat{p}_t = \text{softmax}(\mathbf{W}_{\text{out}}^{\text{class}}\mathbf{y}_t)$ and $y_t \in \{0, 1, 2\}$.

Closed-form gradient (softmax + cross-entropy):

$$\frac{\partial \mathcal{L}_{\text{Dir}}}{\partial \mathbf{o}_t^{\text{out}}} = \hat{p}_t - \mathbf{1}_{y_t}$$

Class imbalance: Direction classes are typically imbalanced (e.g., UP dominates in bull markets). Use inverse-frequency class weights:

$$w_c = \frac{N}{3 \cdot N_c}$$

where N_c is the count of class c in the training set. Apply via weighted cross-entropy: $\mathcal{L}_{\text{Dir}} = -\frac{1}{T}\sum_t w_{y_t} \log \hat{p}_t[y_t]$.

6.3 Sharpe Ratio Loss

The Sharpe ratio measures risk-adjusted returns:

$$\text{SR} = \frac{\mu(\hat{r})}{\sigma(\hat{r})} \cdot \sqrt{252}$$

where $\mu = \frac{1}{T}\sum \hat{r}_t$, $\sigma^2 = \frac{1}{T-1}\sum(\hat{r}_t - \mu)^2$.

The loss is $\mathcal{L}_{\text{SR}} = -\text{SR}$. Its gradient involves **all** predictions through μ and σ:

$$\frac{\partial \mathcal{L}_{\text{SR}}}{\partial \hat{r}_t} = -\frac{\sqrt{252}}{T} \cdot \frac{1}{\sigma} + \frac{\sqrt{252}}{T-1} \cdot \frac{\mu}{\sigma^3}(\hat{r}_t - \mu)$$

where the T^{-1} term comes from $\partial\mu/\partial\hat{r}_t$ and the $(T-1)^{-1}$ term from $\partial\sigma^2/\partial\hat{r}_t$ (unbiased variance). For large T, $T \approx T-1$ and the simpler form $-\frac{\sqrt{252}}{T}\big(\frac{1}{\sigma} - \frac{\mu}{\sigma^3}(\hat{r}_t - \mu)\big)$ is an acceptable approximation.

This is a **global** gradient that couples all time steps, requiring full backpropagation through the SSM sequence.

6.4 Pinball Loss (Quantile Regression)

For Value-at-Risk (VaR) estimation at quantile τ:

$$\mathcal{L}_\tau = \frac{1}{T}\sum_{t=1}^{T} \rho_\tau(r_t - \hat{q}_t^{(\tau)})$$

where $\rho_\tau(u) = u \cdot (\tau - 1_{u<0})$.

Common quantiles: $\tau = 0.05$ (95% VaR), $\tau = 0.01$ (99% VaR), $\tau = 0.5$ (median), $\tau = \{0.05, 0.25, 0.5, 0.75, 0.95\}$ (full distribution).

6.5 Composite Loss

The total loss combines all objectives:

$$\mathcal{L} = \lambda_1\mathcal{L}_{\text{MSE}} + \lambda_2\mathcal{L}_{\text{SR}} + \lambda_3\mathcal{L}_{\text{Dir}} + \sum_\tau \lambda_{4,\tau}\mathcal{L}_\tau + \lambda_5\mathcal{L}_{\text{Reg}}$$

6.6 Learnable Loss Weights (Uncertainty Weighting)

Following Kendall et al. (2018), learning task uncertainty weights stabilizes multi-task training:

$$\mathcal{L} = \sum_i \frac{1}{2}\exp(-s_i) \cdot \mathcal{L}_i + \frac{1}{2}s_i$$

where $s_i = \log\sigma_i^2$ is the learnable log-variance for task i.

Loss warmup schedule: The Sharpe gradient dominates early training when predictions are noisy (σ large), suppressing local losses (MSE, directional). We recommend:

1. Train with $\mathcal{L}_{\text{MSE}}$ only for 500–2000 steps (warmup).
2. Phase in $\mathcal{L}_{\text{SR}}$ with a linear schedule: $\lambda_2(t) = \lambda_2 \cdot \min(1, t/\tau_{\text{warmup}})$.
3. Phase in $\mathcal{L}_{\text{Dir}}$ and $\mathcal{L}_\tau$ similarly after MSE converges.

7 Backpropagation Through the Selective Scan

Mamba backpropagation has two gradient paths: through the SSM recurrence and through the input-dependent parameter projections.

7.1 Gradient Flow

Given $\delta_t^{\text{out}} = \partial\mathcal{L}/\partial\mathbf{o}_t^{\text{out}}$:

$$\frac{\partial\mathcal{L}}{\partial\mathbf{y}_t} = \mathbf{W}_{\text{out}}^\top \delta_t^{\text{out}} \tag{21}$$

$$\frac{\partial\mathcal{L}}{\partial\mathbf{y}_{\text{ssm},t}} = \frac{\partial\mathcal{L}}{\partial\mathbf{y}_t} \odot \text{SiLU}(\mathbf{z}_2) \tag{22}$$

$$\frac{\partial\mathcal{L}}{\partial\mathbf{z}_2} = \frac{\partial\mathcal{L}}{\partial\mathbf{y}_t} \odot \mathbf{y}_{\text{ssm},t} \odot \text{SiLU}'(\mathbf{z}_2) \tag{23}$$

$$\frac{\partial\mathcal{L}}{\partial\mathbf{h}_t} \mathrel{+}= \delta_t^{\text{ssm}} \cdot \mathbf{C}_t \tag{24}$$

$$\frac{\partial\mathcal{L}}{\partial\mathbf{C}_t} = \mathbf{h}_t \cdot (\delta_t^{\text{ssm}})^\top \tag{25}$$

$$\frac{\partial\mathcal{L}}{\partial\mathbf{h}_{t-1}} \mathrel{+}= \frac{\partial\mathcal{L}}{\partial\mathbf{h}_t} \odot \overline{\mathbf{A}}_t \tag{26}$$

$$\frac{\partial\mathcal{L}}{\partial\overline{\mathbf{A}}_t} = \frac{\partial\mathcal{L}}{\partial\mathbf{h}_t} \odot \mathbf{h}_{t-1} \tag{27}$$

$$\frac{\partial\mathcal{L}}{\partial\overline{\mathbf{B}}_t} = \frac{\partial\mathcal{L}}{\partial\mathbf{h}_t} \odot \mathbf{x}'_{\text{conv},t} \tag{28}$$

7.2 Input-Dependent Parameter Gradients

The gradients flow through Δ_t, $\mathbf{B}_t$, $\mathbf{C}_t$:

$$\frac{\partial\mathcal{L}}{\partial\Delta_t} = \frac{\partial\mathcal{L}}{\partial\overline{\mathbf{A}}_t} \odot \overline{\mathbf{A}}_t \odot \mathbf{A} + \frac{\partial\mathcal{L}}{\partial\overline{\mathbf{B}}_t} \odot \mathbf{B}_t^\top \tag{29}$$

$$\frac{\partial\mathcal{L}}{\partial\mathbf{B}_t} = \Delta_t \cdot \frac{\partial\mathcal{L}}{\partial\overline{\mathbf{B}}_t} \tag{30}$$

$$\frac{\partial\mathcal{L}}{\partial\mathbf{x}'_{\text{conv},t}} = \mathbf{W}_\Delta^\top \left(\frac{\partial\mathcal{L}}{\partial\Delta_t} \odot \sigma'(\cdot) \right) + \mathbf{W}_B^\top \frac{\partial\mathcal{L}}{\partial\mathbf{B}_t} + \mathbf{W}_C^\top \frac{\partial\mathcal{L}}{\partial\mathbf{C}_t} \tag{31}$$

7.3 Parameter Gradient Accumulation

All parameter gradients are accumulated over time steps:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{\text{in}}} = \sum_{t=1}^{T} \frac{\partial \mathcal{L}}{\partial \mathbf{z}_t} \mathbf{x}_t^\top, \quad \frac{\partial \mathcal{L}}{\partial \mathbf{W}_{\text{out}}} = \sum_{t=1}^{T} \delta_t^{\text{out}} \mathbf{y}_t^\top \tag{32}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{\Delta}} = \sum_{t=1}^{T} \frac{\partial \mathcal{L}}{\partial \Delta_t} \sigma'(\cdot) (\mathbf{x}'_{\text{conv},t})^\top, \quad \frac{\partial \mathcal{L}}{\partial \mathbf{W}_{B}} = \sum_{t=1}^{T} \frac{\partial \mathcal{L}}{\partial \mathbf{B}_t} (\mathbf{x}'_{\text{conv},t})^\top \tag{33}$$

The gradient for $\mathbf{A}_{\log}$ uses $\mathbf{A} = \exp(\mathbf{A}_{\log})$:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{A}_{\log}} = \frac{\partial \mathcal{L}}{\partial \mathbf{A}} \odot \mathbf{A}$$

Key insight: The diagonal $\overline{\mathbf{A}}_t \in (0, 1)$ structure ensures that $\partial \mathbf{h}_t / \partial \mathbf{h}_{t-1} = \overline{\mathbf{A}}_t$ is an element-wise product, avoiding the vanishing/exploding gradient problem of full-matrix RNNs.

8 Training Pipeline

8.1 Data Preparation

Raw OHLCV prices are transformed to stationary features:

1. **Returns:** $r_t = (p_t - p_{t-1})/p_{t-1}$ (simple return)
2. **Normalization:** Rolling z-score with trailing 252-day window
3. **Technical indicators:** RSI(14), Bollinger bands (20, 2), volume ratio, realized volatility (21-day)
4. **Sequence construction:** Sliding window of length T, input $\{\mathbf{x}_{t-T+1}, \ldots, \mathbf{x}_t\}$, target r_{t+1}

8.2 Walk-Forward Validation

Financial time series require temporal integrity:

```
Train[t0, t1] → Purge[t1, t1+P] → Val[t1+P, t2] →
Train[t0+K, t1+K] → Purge[t1+K, t1+K+P] → Val[t1+K+P, t2+K] → ...
```

where K is the walk-forward step size (e.g., 21 trading days) and P is a *purge period* of length $T + h$ (T = sequence length, h = forecast horizon). The purge prevents look-ahead leakage from the last training sequence (which uses data up to t_1) into the validation period. Additionally, all z-score normalization statistics must be computed on the training fold only – never using validation or test data.

8.3 Hyperparameter Selection

8.4 Monitoring

Key metrics during training:

- **Validation Sharpe ratio:** Primary metric. Declining → overfitting.
- **Directional accuracy:** Percentage of correct direction predictions.
- **Gradient norm:** $\|\nabla\|_2$; consistent growth → instability.
- $\overline{\Delta}_t$ **mean:** Near-zero → "freezing" state; large → instability.
- $\overline{\mathbf{A}}_t$ **evolution:** Regime changes should correspond to shifts.

Table 3: Recommended hyperparameters for CTM

Hyperparameter	Recommended Value
d_{model} (hidden dimension)	64–128
d_{state} (SSM state)	16 (Mamba-1), 64 (Mamba-2)
d_{conv} (conv kernel)	3–5
T (sequence length)	30–120 (1–4 months)
N (Mamba layers)	2–6
Optimizer	AdamW ($\beta_1 = 0.9, \beta_2 = 0.95$)
Learning rate	3×10^{-4} (2K warmup + cosine decay)
Weight decay	0.1 (non-embedding params)
Gradient clip norm	1.0
Loss weights ($\lambda_1 : \lambda_2 : \lambda_3 : \lambda_4$)	1:0.5:1:0.1

9 Scaling Strategies

9.1 Multi-Asset CTM

For N stocks, the output head expands to $\mathbf{W}_{\text{out}} \in \mathbb{R}^{N \times d_{\text{model}}}$:

$$\hat{\mathbf{r}}_t = \mathbf{W}_{\text{out}} \mathbf{y}_t \in \mathbb{R}^N$$

The loss becomes a portfolio-level objective:

$$\mathcal{L}_{\text{portfolio}} = -\text{SR}_{\text{portfolio}} + \lambda \cdot \text{Turnover penalty}$$

9.2 Graph-Enhanced CTM (SAMBA Pattern)

For cross-stock correlation modeling, add a graph convolution layer after Mamba:

$$\mathbf{x}^{(g)} = \sum_{k=0}^{K} \theta_k \mathbf{T}_k(\tilde{\mathbf{L}}) \mathbf{X}$$

where $\mathbf{T}_k$ are Chebyshev polynomials of the normalized Laplacian, and the graph adjacency is learned via a Gaussian kernel:

$$\mathbf{A}_{ij} = \exp\left(-\frac{\|\mathbf{h}_i - \mathbf{h}_j\|^2}{2\sigma^2} \right)$$

9.3 Model Scaling Dimensions

Table 4: Scaling dimensions from μP-SSM research

Parameter	Effect of Increase	Recommended Range
d_{model}	Quadratic param growth $O(3ED^2)$	64–512
$d_{\text{state}} = N$	Higher memory, slower scan	16–128
Expansion E	More params/layer	2
Sequence T	Better long-context	30–504
Layers	Deeper representations	2–8

9.4 Production Deployment

For live trading:

- **Inference:** $\mathcal{O}(1)$ per-step state update; no history caching
- **CPU optimization:** Use OpenBLAS/MKL for matrix ops; batch predictions
- **Streaming:** Maintain $\mathbf{h}_{t-1}$ between batch inferences
- **Quantization:** INT8/FP16 for inference (Mamba-3 supports natively)
- **Ensemble:** Train multiple seeds, average predictions for stability

10 Verification and Debugging

10.1 Numerical Gradient Checking

Always verify backpropagation:

$$\frac{\partial \mathcal{L}}{\partial \theta_i} \approx \frac{\mathcal{L}(\theta + \epsilon e_i) - \mathcal{L}(\theta - \epsilon e_i)}{2\epsilon}, \quad \epsilon = 10^{-4}$$

Acceptance criterion:

$$\frac{\|\mathbf{g}_{\text{analytic}} - \mathbf{g}_{\text{numeric}}\|_2}{\max(\|\mathbf{g}_{\text{analytic}}\|_2, \|\mathbf{g}_{\text{numeric}}\|_2)} < 10^{-5}$$

Verify every parameter: $\mathbf{W}_{\text{in}}, \mathbf{W}_{\text{conv}}, \mathbf{b}_{\text{conv}}, \mathbf{A}_{\log}, \mathbf{D}, \mathbf{W}_{\Delta}, \mathbf{b}_{\Delta}, \mathbf{W}_B, \mathbf{W}_C, \mathbf{W}_{\text{out}}$.

10.2 Common Issues

Table 5: Common debugging scenarios

Symptom	Likely Cause
Loss not decreasing	Learning rate wrong; gradient sign error
NaN loss	Δ negative (softplus overflow); exp overflow
Sharpe not improving	MSE vs. Sharpe gradient conflict; λ imbalance
All gradients zero	Forgot to accumulate over T; init issue
$\overline{\Delta}_t$ all zero	$\mathbf{b}_{\Delta}$ init error; softplus bug
$\mathbf{h}_t$ explosion	$\mathbf{A}$ init positive; $\overline{\mathbf{A}}_t > 1$
Numerical check fails	Wrong/missing gradient term in backprop

11 Implementation Roadmap

11.1 Phase A: Single-Stock CTM (1–2 weeks)

- Implement MambaBlock with selective scan (PyTorch)
- Add Conv-Temporal frontend (causal conv + decomposition)
- Implement composite loss (MSE + Sharpe + Directional + Pinball)
- Train on single stock (SPY/AAPL, 10+ years daily data)
- Target: Validation Sharpe > 0.5, directional accuracy $> 55\%$

11.2 Phase B: Multi-Asset with Walk-Forward (1–2 months)

- Multi-output head for N stocks
- RMSNorm and residual connections per layer
- Walk-forward cross-validation
- Transaction costs integrated into Sharpe loss
- $d_{\text{model}} \to 256$, $N \to 20$–100 stocks
- Target: Portfolio Sharpe > 1.0 net of costs

11.3 Phase C: Production Quant System (3–6 months)

- GPU acceleration (CUDA/MPS/OpenCL)
- Mixed precision (BF16/FP16)
- Distributed training for 500+ stocks
- Live data pipeline (Polygon/Bloomberg/IBKR)
- Backtesting engine integration
- Target: Information ratio > 1.5

12 Conclusion

The CTM architecture provides a principled, scalable framework for stock prediction that combines the strengths of:

- **Convolution:** Local pattern extraction with causal integrity
- **Temporal decomposition:** Multi-scale trend/seasonal separation
- **Mamba:** Linear-complexity long-range sequence modeling
- **Quantitative losses:** Direct optimization of trading metrics

With $\mathcal{O}(T)$ training complexity and $\mathcal{O}(1)$ per-step inference, CTM is uniquely suited for both research prototyping and production trading systems.

A Notation Reference

Symbol	Meaning	Shape
$\mathbf{x}_t$	Input feature vector	$d_x \times 1$
r_t	Actual return	scalar
$\hat{r}_t$	Predicted return	scalar
$\mathbf{z}$	Projected input	$(2d_{\text{model}}) \times 1$
$\mathbf{z}_1$	SSM branch	$d_{\text{model}} \times 1$
$\mathbf{z}_2$	Gate branch	$d_{\text{model}} \times 1$
$\mathbf{x}'_{\text{conv},t}$	Convolution output	$d_{\text{model}} \times 1$
Δ_t	Discretization step size	$d_{\text{model}} \times 1$
$\mathbf{A}$	SSM diagonal matrix	$d_{\text{model}} \times 1$
$\overline{\mathbf{A}}_t$	Discretized $\mathbf{A}$	$d_{\text{model}} \times 1$
$\mathbf{B}_t$	Input matrix	$d_{\text{state}} \times 1$
$\mathbf{C}_t$	Output matrix	$d_{\text{state}} \times 1$
$\mathbf{D}$	Skip connection	$d_{\text{model}} \times 1$
$\mathbf{h}_t$	SSM hidden state	$d_{\text{model}} \times d_{\text{state}}$
$\mathbf{y}_{\text{ssm},t}$	SSM output	$d_{\text{model}} \times 1$
$\mathbf{y}_t$	Gated output	$d_{\text{model}} \times 1$
$\mathcal{L}$	Total loss	scalar
$\odot$	Element-wise multiplication	–
η	Learning rate	scalar

Table 6: Summary of mathematical notation